

Introduction to Machine Learning & Decision Trees

Maciej Piasecki & Halina Kwaśnicka

Learning

○ You know what does **learning** mean, so let's try to define ...

Learning

- The first required aspect: **change**
- Forgetting is also change $\Rightarrow$ **advantageous**
- A well-fed animal gains an advantage $\Rightarrow$ **autonomous**
- A random change is autonomous $\Rightarrow$ **a result of experience**
- **Change** (must be observed), but:
 - **advantageous** (forgetting is also a change),
 - **autonomous** (better feeding improves effectiveness),
 - **on the basis of experience** (random change is autonomous).

Concept of Learning

- Learning system is a system in which an autonomous change proceeds on the basis of experience, and this change causes better functionality of the system
- Learning – a fuzzy concept (a change can be objectively stated, but the rest are fuzzy, conventional)

Learning System

"A computer program is said to **learn** from the experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E "

T.M. Mitchell „Machine Learning“

Learning System – Examples

○ Program for mail categorisation

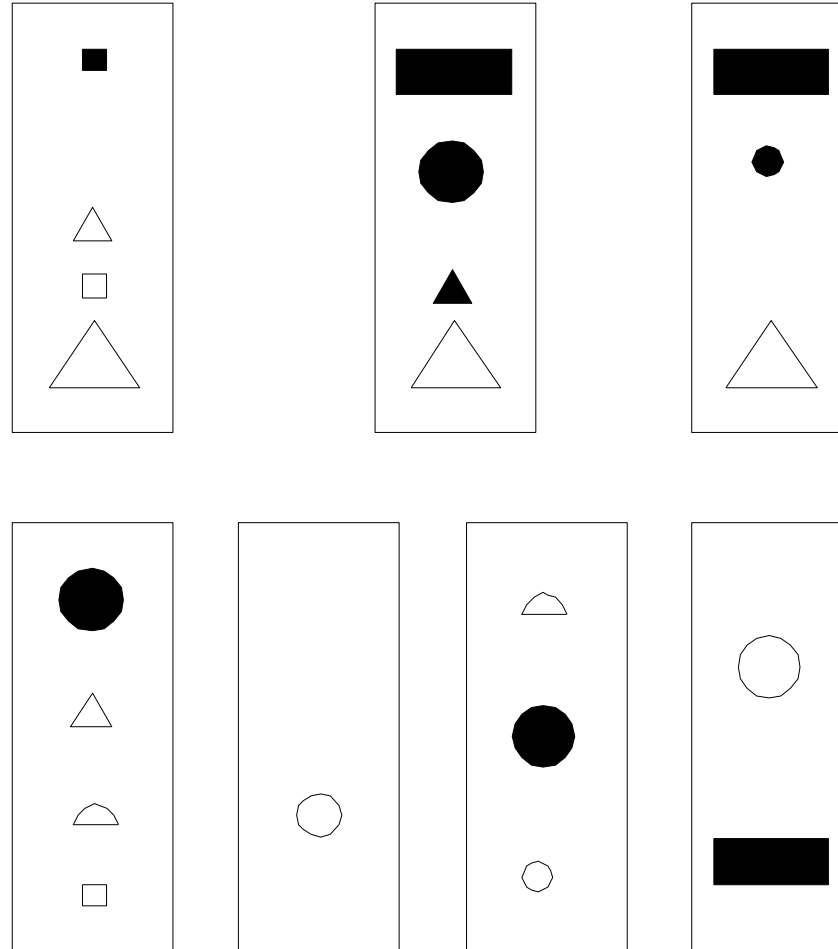
- Task T:
taking decisions on categories of incoming mails
 - * (e.g. question, report, pertaining department, etc.)
- Measure of performance P:
the number of documents which have been delivered (classified) correctly
vs the number documents left without the decision
- Experiences E : documents described by labels

○ Vehicle control problem

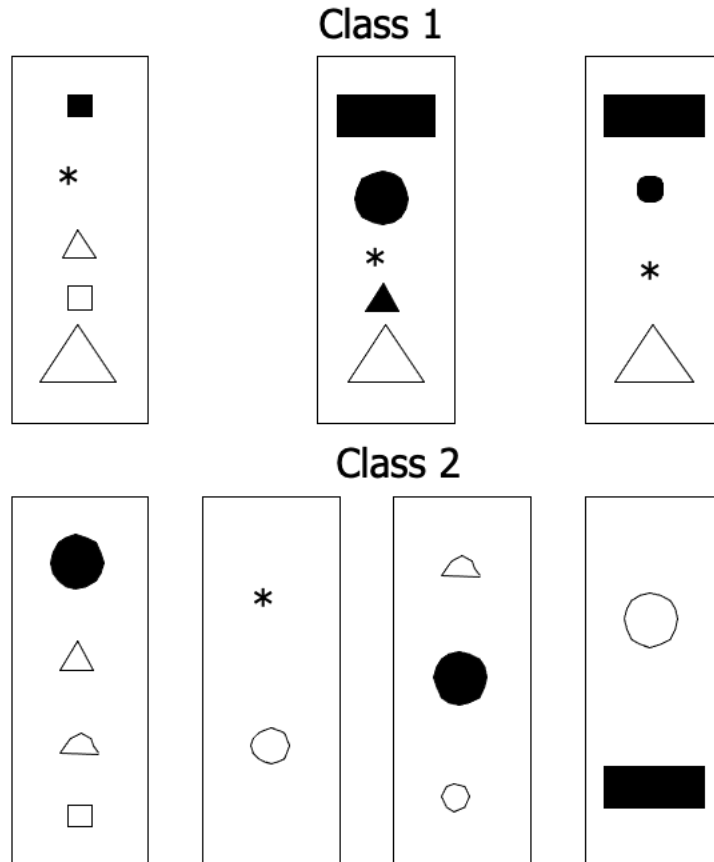
- Class of tasks T:
movement on public roads in four directions using only visual sensors
- Measure of performance P:
length of the road covered without mistakes on average
- Experiences E:
a sequence of images and commands given by a human driver (collected
by observation)

○ Generative Neural Models: Large Language Models and Stable Diffusion Models (for images)

Inductive learning



Inductive learning



- R1: IF a rectangle contains a small black shape THEN Class 1
- R2: IF a rectangle contains five figures OR a small black circle THEN Class 1
- R3: IF a rectangle contains two figures OR a white 'crescent' shape THEN Class 2.
- ...

Deduction vs Induction

○ **Deduction** – inference from general to specific (fact) – top-down:

- Known: *All dogs have four legs.*
- We know that: *Patcher is my dog.*
- We deduct: *Patcher has four legs.*

○ **Induction** – inference from specific (facts) to general – bottom-up:

- We see:
 - * one leaf is green and thin,
 - * next leaf is green, thin and oval,
 - * next – green, thin, and oblong, ...
- We induct:
 - * All leaves are green and thin.

Basic Concepts

○ Paradigm:

- Inductive learning
- Case Based Reasoning

○ Data:

- Symbolic: strings of symbols with assigned interpretation
- Sub-symbolic: information whose elements do not have interpretation

○ Character of the input

- Unsupervised learning – mainly clustering
- Supervised learning
- Reinforcement learning

○ Tasks

- Approximation – mapping objects onto a set of real numbers
 - * Regression — mapping object on a measure of relatedness or similarity
 - * target function: objects → numerical values
- Classification – deciding about assignment of objects to some categories
 - * target function: objects → symbols

General Problems with Learning

- Basic assumption – the **Induction Learning Hypothesis**:
 - Any hypothesis found that approximates the target function well on a large part of the training data set will also approximate the target function for unobserved examples.
- Definition
 - The **bias of an inductive learning algorithm** is factors that decide on the choice of one hypothesis from among the set of hypotheses that are admissible from the perspective of the learning goal.

Training and test sets

- The assumption that sets of examples are independent of hypotheses is unfulfilled, because the hypotheses are generated on the basis of the examples
- Why and how many separate sets?
 - a training set
 - a development set (called also tuning or even validation)
 - a test set (validation)
 - * kept aside (hidden/secret) during the training (development)
- **k-fold Cross-validation**
 - random dividing of the labelled data set into k subsets of similar size and distribution
 - $k-1$ subsets (folds) for training, 1 for testing
 - average from the results of the k experiments

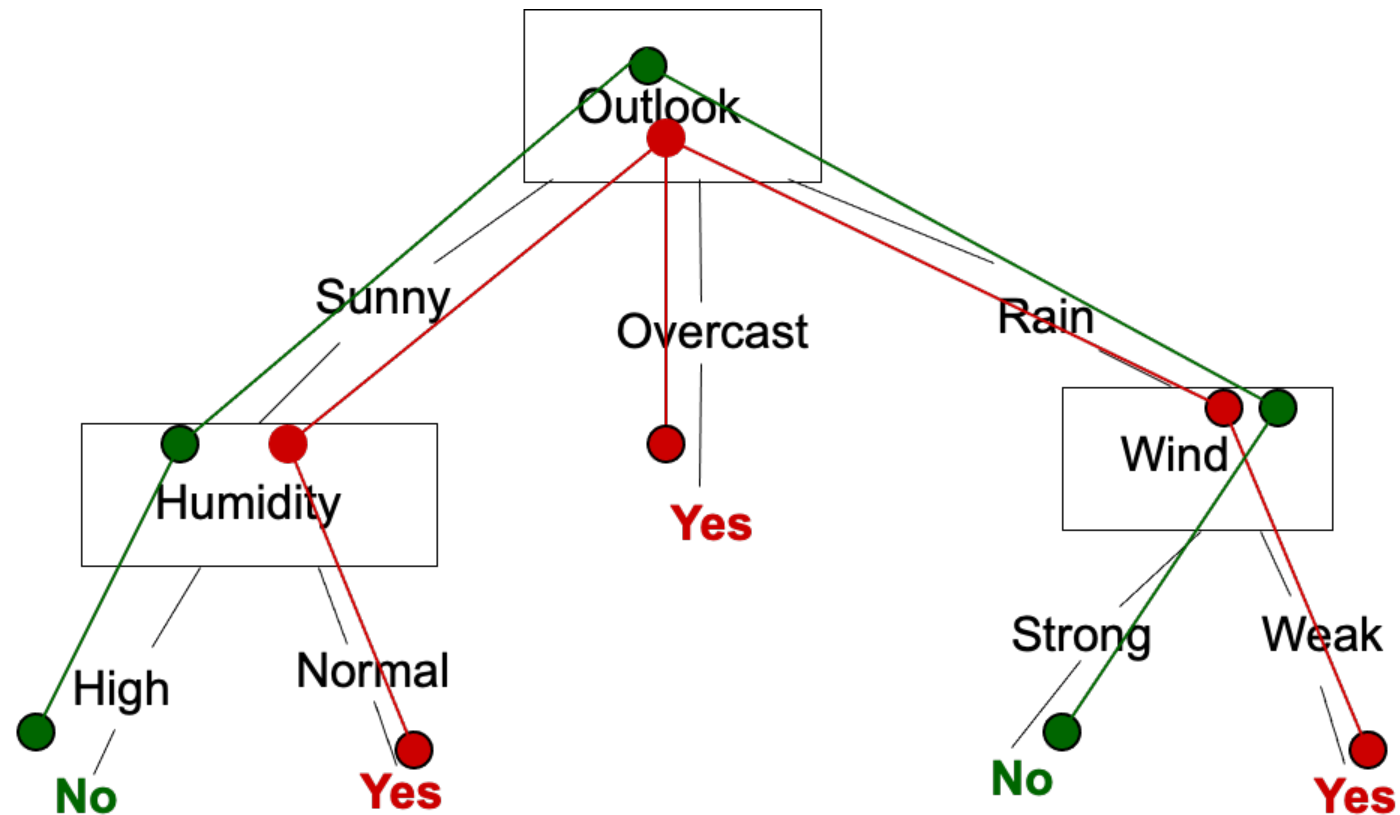
Time to learn ...

DECISION TREES INDUCTION

Decision Tree Induction

- Decision Tree Induction = Decision Tree Learning
- Decision Tree structure and learning it from data
 - are widely spread ideas in Machine Learning
 - could be traced in many algorithms, also contemporary, e.g. Random Forest,
 - applied in many practical approaches to inductive inference
 - known for their self-explainability — a rare feature in ML
- Each node specifies a test of some attribute of an instance ...

Example: a decision tree



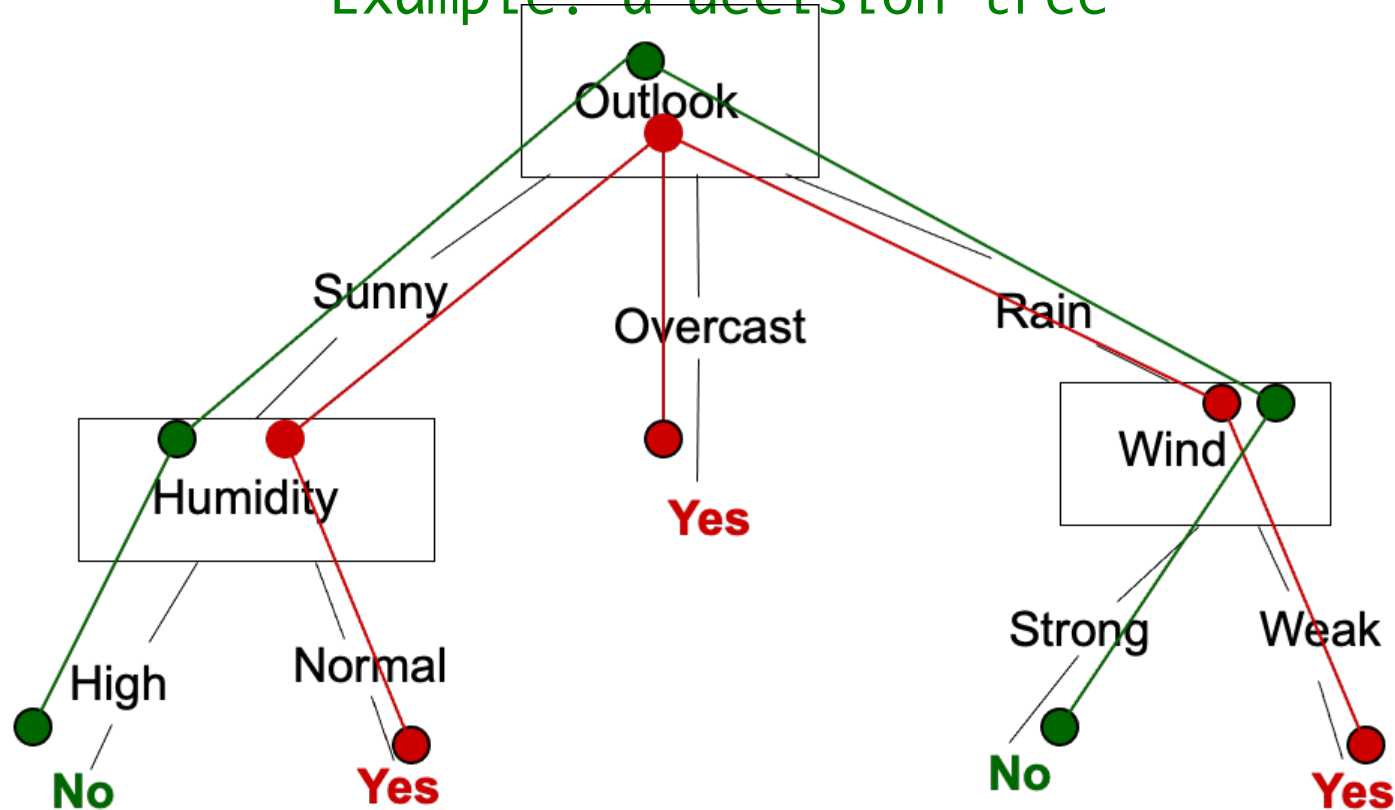
Decision Tree Induction

- Each node specifies a test of some attribute of an instance
 - outgoing arcs correspond to the possible values of the node attribute
- Decision trees classify instances
 - by sorting them down the tree from the root to some leaf node
 - all non-leaf nodes contain attribute names,
 - leaf nodes contain class values,
 - each branch corresponds to the attribute value
- Algorithms:
 - ID3, ASSISTANT, C4.5, C5.0, ...
 - inheriting the idea: Random Forest,

Illustrative example

Day	Outlook	Temperature	Humidity	Wind	Decision
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	Yes
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Example: a decision tree



- (Outlook = Sunny $\wedge$ Humidity = Normal)
 $\vee$ (Outlook = Overcast) $\vee$ (Outlook = Rain $\wedge$ Wind = Weak)
 $\rightarrow$ Decision=Yes
- (Outlook = Sunny $\wedge$ Humidity = High)
 $\vee$ (Outlook = Rain $\wedge$ Wind = Strong) $\rightarrow$ Decision=No

Basic ID3 algorithm (Quinlan 1986)

- ID3 learns decision trees in a top-down way
- Important question:
which attribute should be tested at the root of the tree?
 - each attribute is evaluated using a statistical test to determine how well it classifies the training examples
 - the best attribute is selected and used as a test at the root node of a tree
- For each possible value of this attribute
 - a root's descendant is created,
 - and the examples during testing are sorted and passed to the appropriate descendant nodes
- The process is recursively repeated
 - using training examples associated with each descendant node
 - to select the best attribute to test at that point in the tree
 - * each attribute is used only once on a path
 - till an unambiguous subset of examples is reached for a given node
 - * the stop condition

The best attribute to classify

Information gain

- a statistical property that measures how well a given attribute separates the training examples according to their target classification

Let's start with a logic target function (two classes):

- $Entropy(S) = -p_{pos} \log_2(p_{pos}) - p_{neg} \log_2(p_{neg})$

Assuming:

- A is a candidate for the tree root attribute
- A is an attribute with n values $A_1, \dots, A_n$

Set S is divided by A into n disjoint subsets $S_1, \dots, S_n$, where each S_i contains examples such that $val(A) = A_i$

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

- where $Values(A)$ is a set of all possible values of A

Entropy – a short reminder

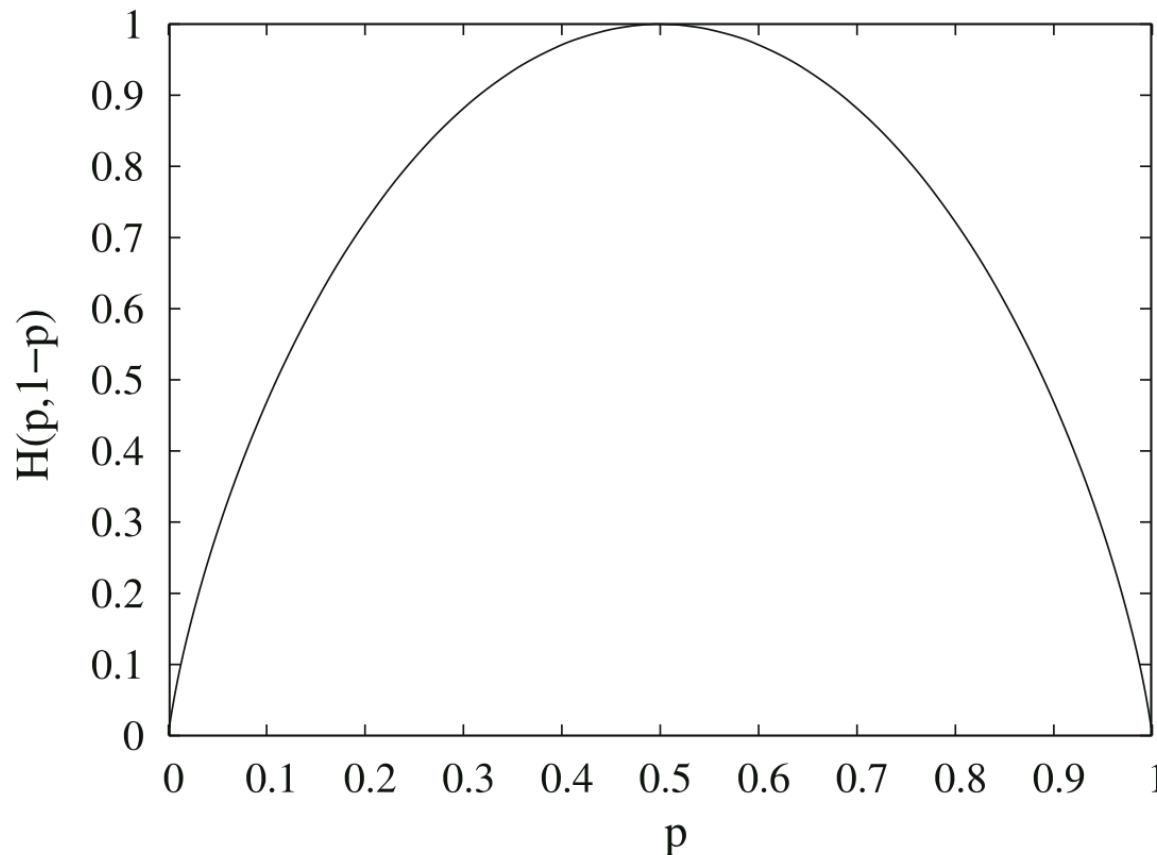


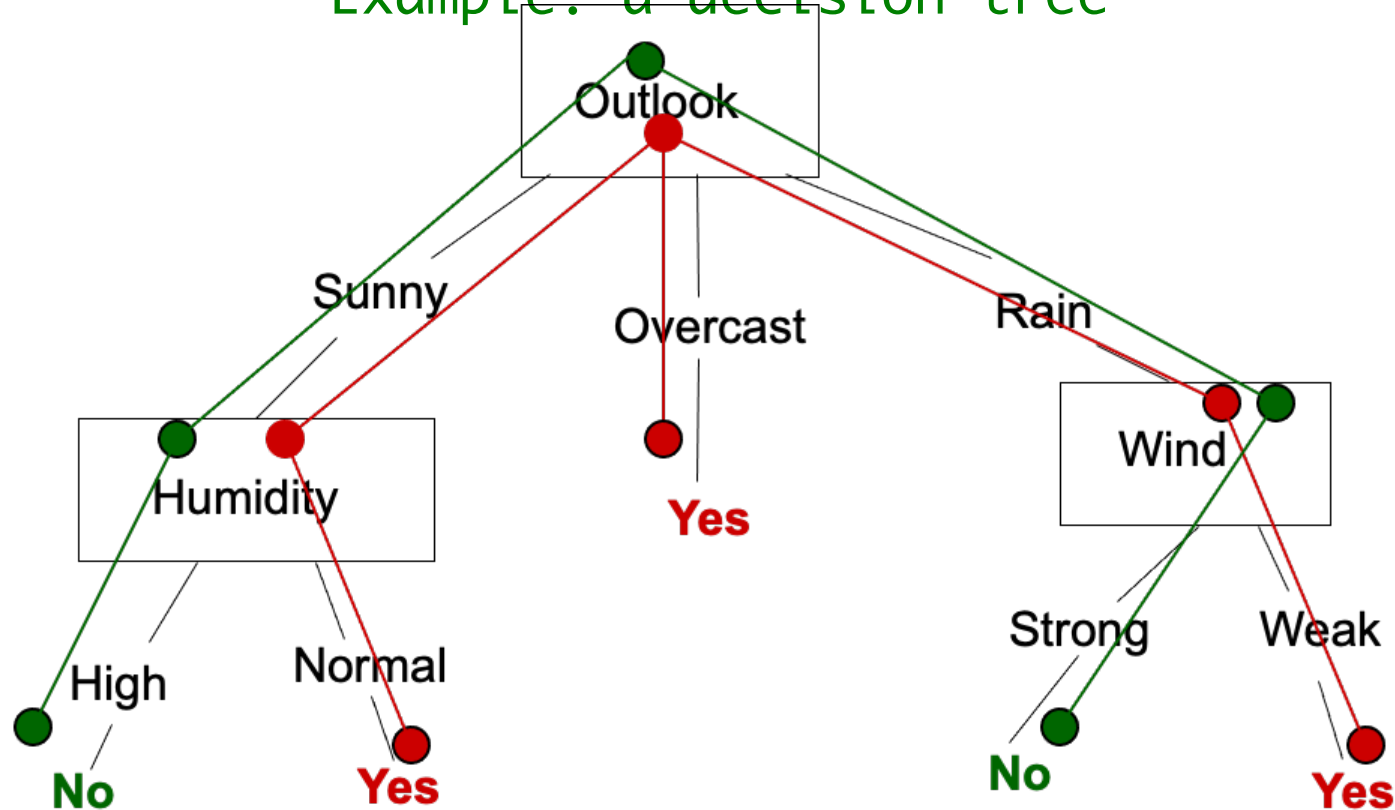
Figure. Entropy for a logical function
(Ertel, str. 202, Fig. 8.24, 2017)

- Entropy equals 0, iff all samples from S belong to the same class
- Entropy equals 1 iff S includes the equal number of positive and negative samples.
- Entropy is less than 1 iff the number of positive samples is different from the number of negative ones.

Illustrative example

Day	Outlook	Temperature	Humidity	Wind	Decision
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	Yes
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Example: a decision tree



- $(\text{Outlook} = \text{Sunny} \wedge \text{Humidity} = \text{Normal}) \vee (\text{Outlook} = \text{Overcast}) \vee (\text{Outlook} = \text{Rain} \wedge \text{Wind} = \text{Weak}) \rightarrow \text{Decision} = \text{Yes}$
- $(\text{Outlook} = \text{Sunny} \wedge \text{Humidity} = \text{High}) \vee (\text{Outlook} = \text{Rain} \wedge \text{Wind} = \text{Strong}) \rightarrow \text{Decision} = \text{No}$

Tree generation

Let $S = [9_{\text{pos}}, 5_{\text{neg}}]$

Entropy(S) = $-p_{\text{pos}}\log_2(p_{\text{pos}}) + -p_{\text{neg}}\log_2(p_{\text{neg}})$
 $= -(9/14) \times \log_2(9/14) + -(5/14) \times \log_2(5/14) = 0.940$

Which attribute should be tested first in the tree?

calculate information gain for each candidate attribute (Outlook, Temperature, Humidity, Wind):

Values(Wind) = {Weak, Strong}: Weak $\Rightarrow [6_{\text{pos}}, 2_{\text{neg}}]$; Strong $\Rightarrow [3_{\text{pos}}, 3_{\text{neg}}]$

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Gain for Wind:

$$\begin{aligned} * E(S) - (8/14) \times E(\text{Weak}) - (6/14) \times E(\text{Strong}) \\ = 0.940 - (8/14) \times 0.811 - (6/14) \times 1.00 = 0.048 \end{aligned}$$

Calculate information gains for all attributes

Gain(S , Outlook) = 0.246

Gain(S , Humidity) = 0.151

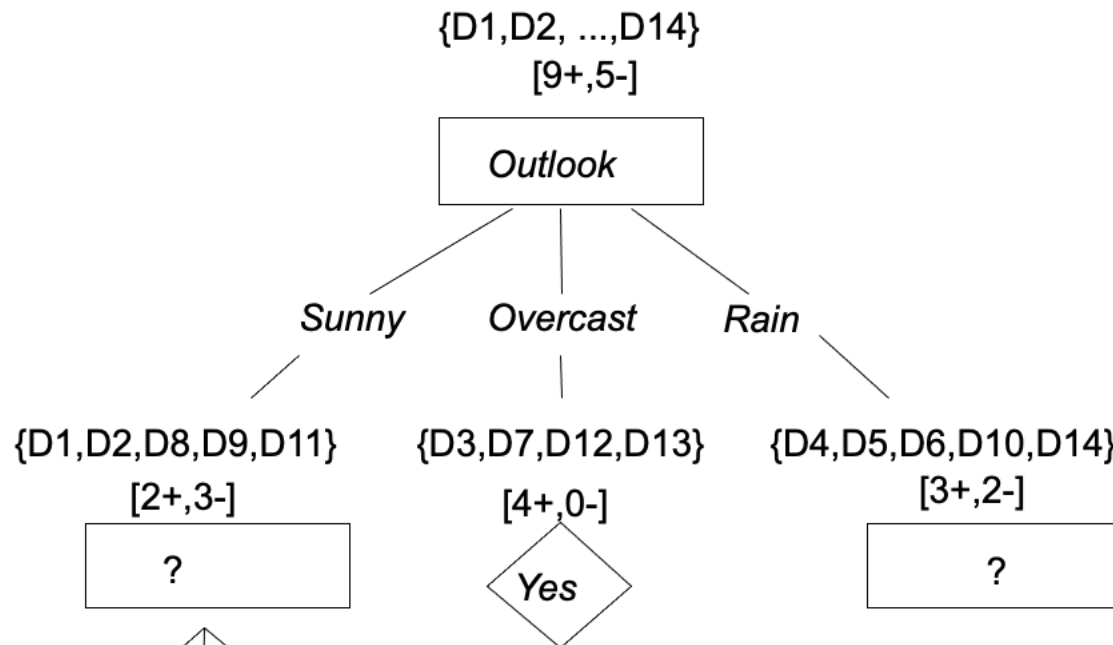
Gain(S , Wind) = 0.048

Gain(S , Temperature) = 0.029

Data Sorted by Outlook

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
D3	Overcast	Hot	High	Weak	Yes
D7	Overcast	Cool	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D10	Rain	Mild	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	NORMAL	WEAK	YES
D11	Sunny	Mild	NORMAL	STRONG	YES

Partial tree



Which attribute should be tested here?

$S_{SUNNY} = \{D1, D2, D8, D9, D11\}$

$\text{Gain}(S_{Sunny}, \text{Humidity}) = 0,970 - (3/5)0,0 - (2/5)0,0 = 0,970$

$\text{Gain}(S_{Sunny}, \text{Temperature}) = 0,970 - (2/5)0,0 - (2/5)0,0 - (1/5)0,0 = 0,570$

$\text{Gain}(S_{Sunny}, \text{Wind}) = 0,970 - (2/5)1,0 - (3/5)0,918 = 0,019$

Pseudocode

ID3(*Examples*, *Target_attribute*, *Attributes*)

Examples are the training examples. *Target_attribute* is the attribute whose value is to be predicted by the tree. *Attributes* is a list of other attributes that may be tested by the learned decision tree. Returns a decision tree that correctly classifies the given *Examples*.

- Create a *Root* node for the tree
- If all *Examples* are positive, Return the single-node tree *Root*, with label = +
- If all *Examples* are negative, Return the single-node tree *Root*, with label = -
- If *Attributes* is empty, Return the single-node tree *Root*, with label = most common value of *Target_attribute* in *Examples*
- Otherwise Begin
 - $A \leftarrow$ the attribute from *Attributes* that best* classifies *Examples*
 - The decision attribute for *Root* $\leftarrow A$
 - For each possible value, v_i , of A ,
 - Add a new tree branch below *Root*, corresponding to the test $A = v_i$
 - Let $Examples_{v_i}$ be the subset of *Examples* that have value v_i for A
 - If $Examples_{v_i}$ is empty
 - Then below this new branch add a leaf node with label = most common value of *Target_attribute* in *Examples*
 - Else below this new branch add the subtree
ID3($Examples_{v_i}$, *Target_attribute*, $Attributes - \{A\}$)
- End
- Return *Root*

Example of practical application

- Problem: recognition of grammatical classes of words in text, e.g. verbs (fin = finite forms) vs nouns (subs = substantive)
- TaKIPi - the first tagger for Polish used in many applications
 - based on a combination of almost 100 decision trees
 - some attributes triggering complex constraints (functions run on text)

Attributes

fin subst : #

!std



isbig(orth[0])

!AdjPRight

!AdjPLeft

!SubstCanIBeSubject

!NumPGenLeft

!PPron3GenLeft

!SieNear

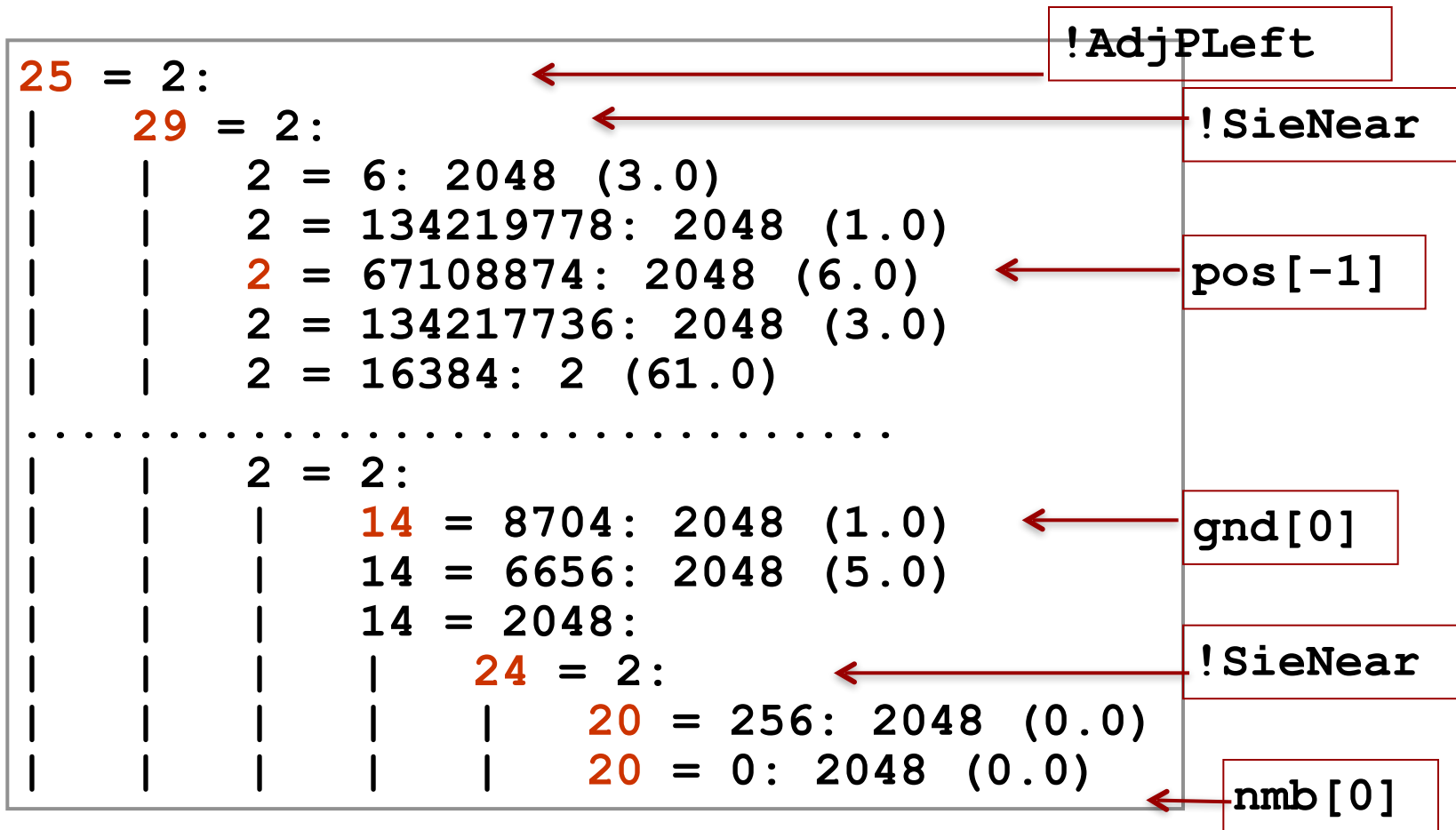
!FinAmIMainVerb

!CasAgrAcrossConjLongStrict

pos[-3]	pos[-2]	pos[-1]
	pos[1]	pos[2]
cas[-3]	cas[-2]	cas[-1]
cas[0]	cas[1]	cas[2]
gnd[-3]	gnd[-2]	gnd[-1]
gnd[0]	gnd[1]	gnd[2]
nmb[-3]	nmb[-2]	nmb[-1]
nmb[0]	nmb[1]	nmb[2]

Example of practical application

- A fragment of a decision tree for the distinction:
verb (fin) vs noun (subst)



Short summary

○ Hypothesis space of all decision trees

- a complete space of finite discrete-valued functions, relative to the available attributes.

○ Single current hypothesis

- during searching the space of decision trees.
- no means to determine how many alternative decision trees are consistent with the training data.

○ No backtracking

- therefore it converges to the local optimum (produces locally optimal solution)
- greedy search with InfoGain as a heuristic function (sic!)

○ All training examples and some statistical properties of the examples are utilised

- smaller sensitivity to errors in an individual example.

Evaluation of a decision tree

○ Random separation of data into two parts:

□ training & testing

- * k-fold cross-validation technique
- * for generating a tree and
- * evaluation of the correctness of object classification (during development of a solution)

□ tuning data

- * selection of parameter values, tree pruning

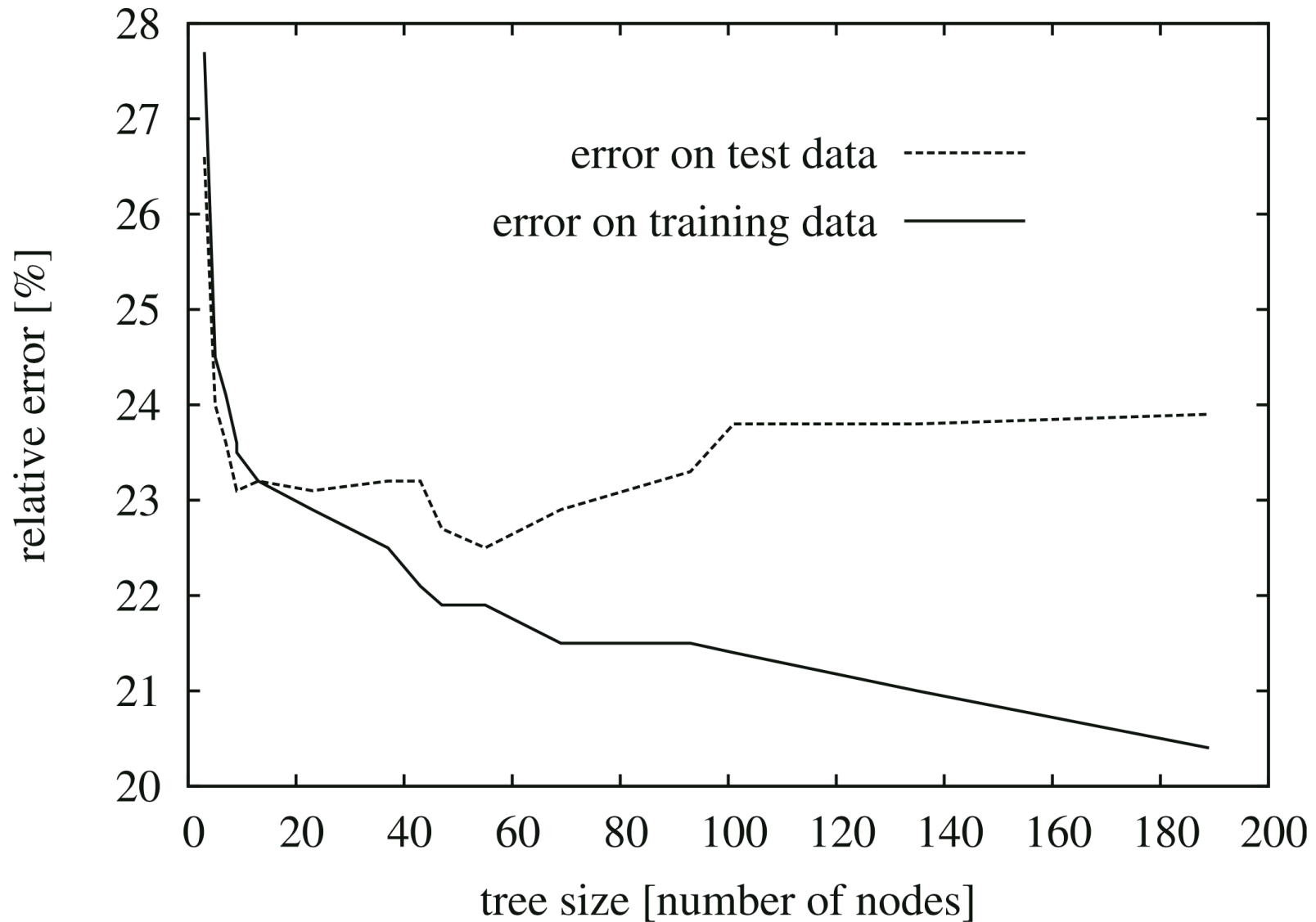
○ Additional set

- held for a final evaluation - **validation set** (held-out data)

Overfitting to data (!)

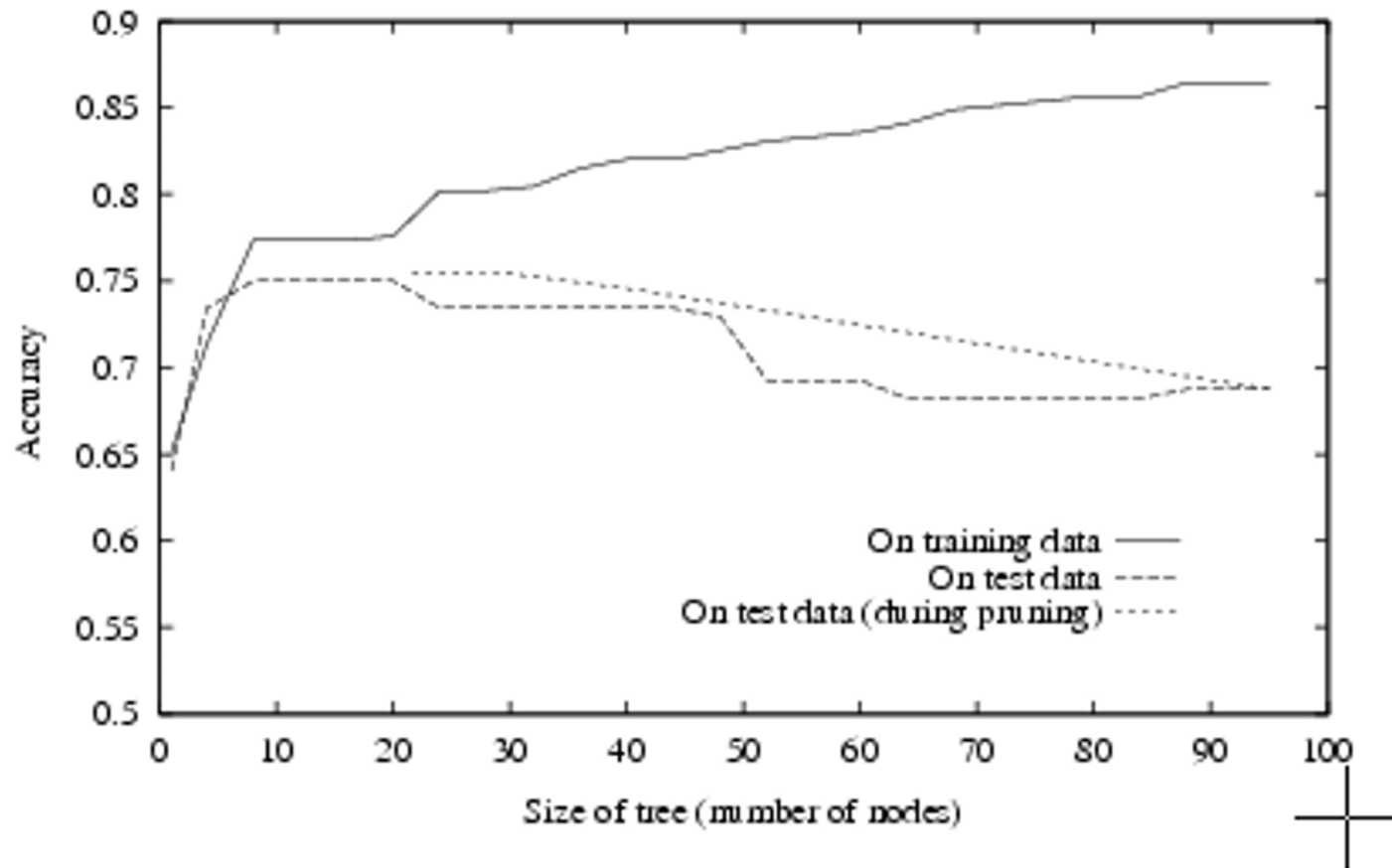
- A hypothesis $h \in H$ is called overfitted to data, if there is an alternative hypothesis $h' \in H$, such that:
 - h expresses a smaller error than h' on the training subset,
 - but h' results in a smaller error than h on the input distribution of cases
- With adding more nodes, its **performance monotonically increases** as measured on the training data
- However, in the case of test data, initially the tree performance is increasing, but later it **starts falling down (!)**

Avoiding Overfitting



(Ertel, str. 211, Fig. 8.27, 2017)

Error reduction by pruning



Preventing overfitting

○ Stopping

- Stop tree expansion before it reaches a point of perfect classification of training data.

○ Pruning

- First, allow the algorithm to overfit to the data, next to prune the tree (post-pruning).

○ Criteria for deciding about an appropriate size for a tree

- approaches:

- * division of data into training and testing subsets,
- * using all training examples during building the tree, and statistical tests in order to decide whether to further expand it,
- * e.g. Quinlan applied chi-square test to decide if further tree expansion would improve its performance on the input distribution of cases, or only on the training data.

Preventing overfitting - criteria for the tree size

- Division of data into training and testing subsets,
 - using all training examples during building the tree, and statistical tests in order to decide whether to further expand it,
 - e.g. Quinlan applied chi-square test to decide if further tree expansion would improve its performance on the input distribution of cases, or only on the training data.
- Explicit tree complexity measure
 - for encoded training data and a decision tree
 - stopping the tree expansion when the encoding is minimal according (Minimum Description Length principle)

Preventing overfitting

- Reduced-error pruning: each node is a candidate to be pruned
- Pruning a node:
 - removing a subtree rooted by the given node
 - turning the node into a leaf
 - assigning the most frequent class in the subtree to the node
- A node is pruned, if the performance of the resulting tree is not worse than the performance of the original one

Attributes with continuous values

- Example:
 - temperature: 40 48 60 72 80 90
 - play_tennis: No No Yes Yes Yes No
- Finding a **threshold** value that expresses the biggest information gain
- Concerning the above example
 - two potential thresholds: $(48+60)/2$ and $(80+90)/2$
 - higher information gain: 54
- Other approaches
 - continuous space is divided into more than two subranges
 - → **quantisation problem**

C4.5 algorithm

- Expanded ID3 algorithm (Quinlan).
- Avoids overfitting to data
- Decides about the depth of a tree
- Pruning error reduction
- Rule-based pruning after a tree has been built
- Support for attributes with continuous values, e.g., temperature
- Improved measure for selecting attributes
- Support for missing attribute values in training data
- Support for attributes with different costs
- Improved computational complexity

Attribute informativeness measure

- Goal: avoiding the effect of the preference for attributes with larger number of possible values
- Information gain (ID3) $\Rightarrow$ relative information gain
 - gain ratio

$$\text{GainRatio}(X) = \frac{(\text{Gain}(X))}{(\text{SplitInfo}(X))}$$

- where $\text{SplitInfo}(X)$ is information gained by clustering objects according to the value of X :

$$\text{SplitInfo}(X) = - \sum_{i=1}^n \left(\frac{|C_i|}{|C|} \right) \log_2 \left(\frac{|C_i|}{|C|} \right)$$

Attributes with continuous values

- For a continuous attribute A with the values $\{v_1, v_2, \dots, v_n\}$ in training data:
 - identify all possible binary divisions according to the subsequent values of v_i , i.e. $\{x: x \leq v_i\}$ vs $\{x: x > v_i\}$
 - information gain or gain ratio is calculated for each division
 - a division maximising the measure is selected
- Continuous attributes (contrary to discrete ones) may appear on several levels of the same tree branch
- For each occurrence a different quantisation may be applied

Literature

- Wolfgang Ertel. Introduction to Artificial Intelligence. Springer, 2017. (available via WUST network)
- Miroslav Kubat. An Introduction to Machine Learning. Springer, 2017. (available via WUST network)
- <https://cis.temple.edu/~ingargio/cis587/readings/id3-c45.html#5>